Modern AI: Large Language Models

What is a Large Language Model (LLM)?

Key question

What are LLMs and where do they come from?

In this part

- What is an LLM?
- Where LLMs fit
- A short history

What is a large language model?

A large language model (LLM) is a very large neural network trained on huge amounts of text to predict the next piece of text. That one skill lets it answer questions, write, summarize, translate and code.

1 Answer
Questions, concepts, error messages.

2 Write
Emails, reports, summaries of long documents.

3 Translate
Between languages, styles and formats.

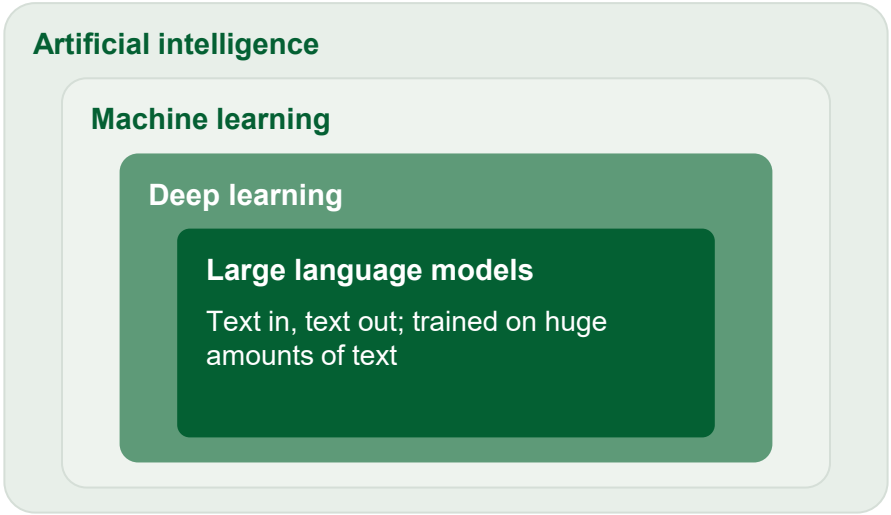
4 Code
Draft, explain and fix code.

Type	Examples
Chat assistants	ChatGPT, Claude, Gemini
Open-weight models	Llama, Mistral, Qwen: can run on an organisation's own computers

One skill: predicting the next piece of text, done extremely well, is enough for all of these tasks.

Where LLMs fit

LLMs are a kind of deep learning model (neural networks with many layers), which is part of machine learning, which is part of artificial intelligence.



Related terms

- **Deep learning:** neural networks with many layers.
- **Foundation model:** pretrained on huge data, adapted to many tasks.
- **Generative AI:** creates new text, images, audio or code.
- **LLM:** a foundation model for language.

Same idea: learning from data, training and testing, and errors that must be checked.

A short history of modern LLMs

Modern LLMs grew from one 2017 architecture, the transformer, scaled up with more data and computing power and then trained to follow instructions.



Transformer

The architecture behind every LLM.

BERT, GPT

First widely used pretrained language models.

GPT-3

Learns new tasks from a few examples in the prompt.

ChatGPT

Instruction-tuned chat brings LLMs to the public.

Multimodal, open

Images and audio; strong open-weight models.

Reasoning, agents

Think before answering; use tools.

Same core idea: predict the next token with a transformer. What changed is scale, instruction tuning and the systems around it.

How do LLMs work?

Key question

What happens between a question and an answer?

In this part

- How it is made
- Tokens
- Attention
- Writing an answer
- Temperature
- Context window

How an LLM is made

An LLM is made in stages: pretraining on huge amounts of text, instruction tuning on example conversations, and feedback training to prefer helpful, safe answers.

1 Pretraining

Read huge amounts of text; predict the next token.

Result: knows language, but only continues text.

2 Instruction tuning

Learn from example conversations written by people.

Result: answers questions, follows requests.

3 Feedback training

People or checkers rate answers; prefer the better ones.

Result: more helpful and safer answers.

Cost: pretraining takes months and huge computing power; that is why most people use existing models rather than train their own.

Tokens

A token is the unit of text an LLM reads and writes: a common word, part of a word, a number or a punctuation mark. Everything an LLM does is counted in tokens.

The sentence “The patient's readmission risk is high.” as tokens



1 Size

1 token is about ¾ of an English word.

2 Cost and limits

Prices and limits are counted in tokens.

3 Odd effects

Words split into pieces, so letter-counting can fail.

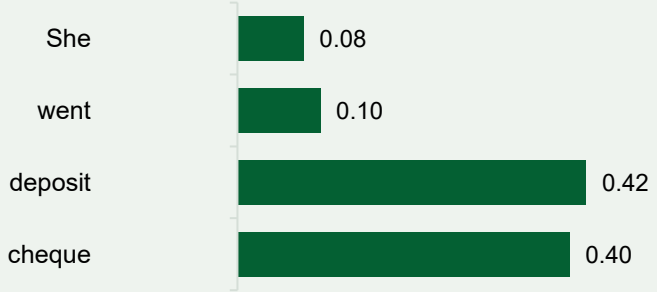
Rule of thumb: 100 tokens is about 75 English words.

Attention: how the model uses context

Attention is the mechanism inside a transformer that lets each word look at every other word and decide which ones matter most for its meaning.

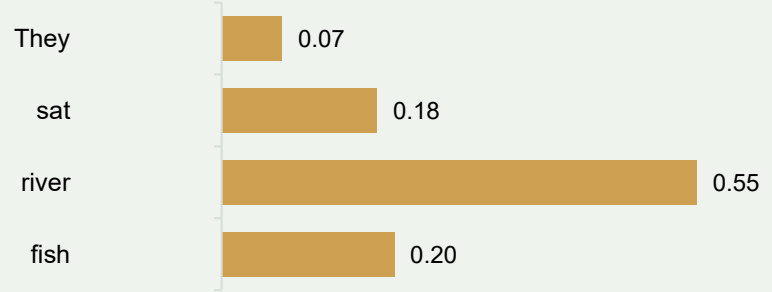
“She went to the bank to deposit a check.”

How much “bank” attends to each word



“They sat on the river bank to fish.”

How much “bank” attends to each word



The idea: each word looks at the others to work out what it means here. The model then reads “bank” as money or as a river.

How an LLM writes an answer

An LLM writes one token at a time: it gives every possible next token a probability, picks one, adds it to the text, and repeats until the answer is complete.

1 Read the text → **2 Attention layers** → **3 Probabilities** → **4 Pick and repeat**

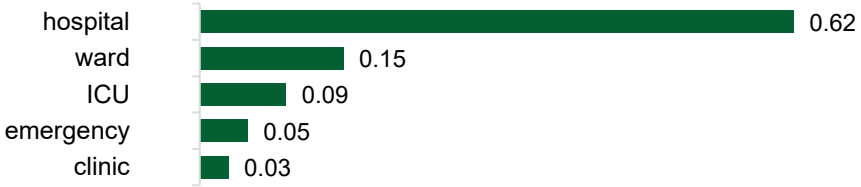
Prompt split into tokens.

Work out what each token means here.

A score for every possible next token.

Add the chosen token; go again.

"The patient was admitted to the ___": next-token probabilities



Two consequences

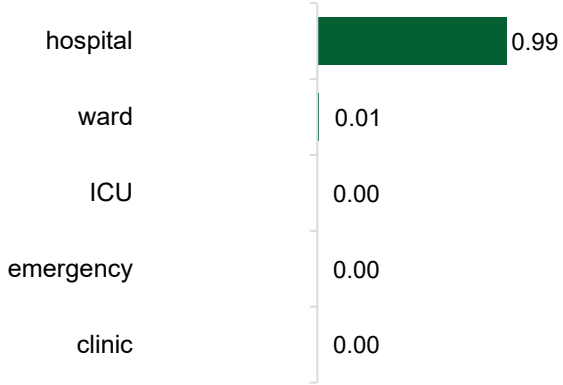
- It produces plausible text; it does not look facts up.
- It writes left to right, so early mistakes carry on.

Scale: a 300-word answer is about 400 rounds of "predict, pick, add".

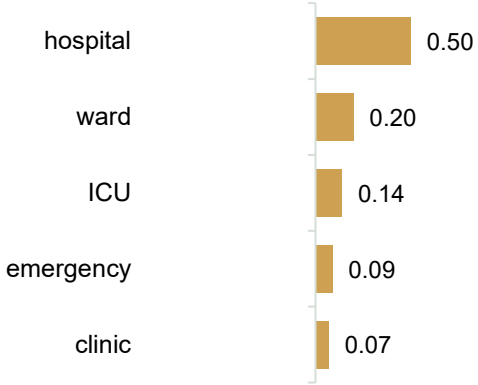
Temperature

Temperature is a setting that controls how the next token is picked from the probabilities: low temperature almost always picks the most likely token; high temperature picks less likely tokens more often.

Low (0.3): consistent



High (1.5): varied



Where is it set?

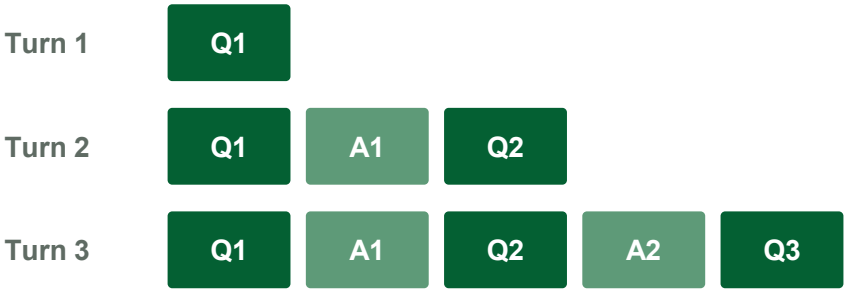
- **API / code:** a parameter you choose, e.g. temperature=0.2.
- **Playgrounds:** a slider.
- **Chat apps:** set for you; usually fixed.
- **Related:** top-p limits choices to the likeliest tokens.

Rule of thumb: low (0 to 0.3) for facts, extraction and code; higher (0.7 to 1) for brainstorming and creative writing.

The context window

The context window is everything the model can read at one time while writing an answer: instructions, the conversation so far, any documents, and the answer itself. Its size is limited and counted in tokens.

What the model reads at each turn of a chat



Q = your question, A = the model's answer. The app re-sends everything each time; when it no longer fits, the oldest parts are dropped or summarised.

Key points

- **Its only memory:** if it is not in the window, the model cannot use it.
- **Fixed size:** often about 200,000 tokens (about a 500-page book); some models over a million.
- **When full:** older parts are cut or summarized, so long chats “forget”.
- **More is not always better:** a cluttered window gives worse, slower answers.

Analogy: a reader at a desk can only use the papers on the desk, and the desk has a fixed size.

Ways to use LLMs

Key question

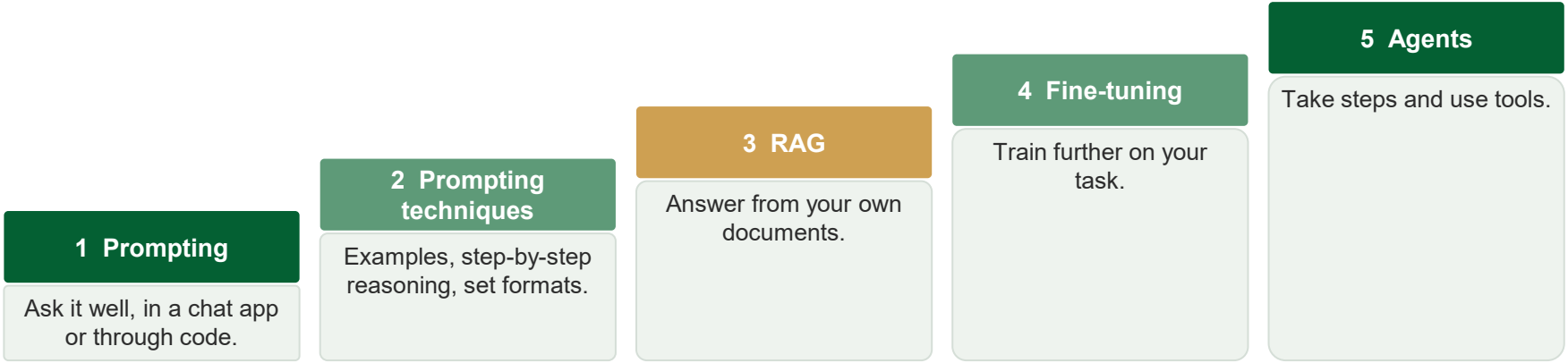
How do we put an LLM to work?

In this part

Overview	Prompting	Techniques	RAG	Fine-tuning	Agents	Choosing

Ways to use an LLM

Ways of using an LLM range from simply asking it questions to building it into systems that search documents, learn a specialized task or take actions with tools.



Start at the bottom: most tasks are solved by good prompting; climb only when it is not enough.

Prompting: Elements of a good prompt

Prompt engineering is designing the input to an LLM so it reliably produces the output you need. A good prompt has four elements: instruction, context, input data and output format.

1. **Instruction:** The task, as a clear action.
2. **Context:** Audience, purpose, rules, background.
3. **Input data:** The material to work on, clearly marked.
4. **Output indicator:** The exact shape of the answer.

Example prompt

Summarize the model results below for the hospital's operations manager.

She is not a data scientist. Use plain language and avoid statistical terms.

Results: <recall 0.85, precision 0.89, 1,500 test patients, weaker for over-80s>

Answer in three bullet points, ending with one recommendation.

Optional extras: a role (“You are an experienced clinical analyst”) and worked examples (few-shot).

Prompting techniques

Prompting techniques are reusable patterns for structuring a request, such as giving examples or asking for step-by-step reasoning.

1 Zero-shot

Just the instruction.

Example: “Classify this review as positive, negative or mixed.”

2 Few-shot

Add a few worked examples to copy.

Example: 3 labelled reviews, then the new one.

3 Chain of thought

Ask for step-by-step reasoning first.

Example: “Work through it step by step, then answer.”

4 Structured output

Ask for a fixed format a program can read.

Example: “Return JSON with fields label and reason.”

Test it: judge a prompt on a fixed set of inputs, not on one impressive answer.

Retrieval-augmented generation (RAG)

Retrieval-augmented generation first searches a trusted collection of documents for passages relevant to the question, then gives those passages to the LLM to answer from.

1 Question	2 Retrieve	3 Add to context	4 Answer with sources
"What is our sepsis escalation policy?"	Search trusted documents for relevant passages.	Put the passages in the prompt.	The LLM answers from them and cites them.

Benefits: uses private and current information, every claim can be traced to a source, and hallucination drops.

Limit: if retrieval finds the wrong passages, the answer is wrong too. Test retrieval quality like any model. The search often uses cosine similarity (lecture 4).

Fine-tuning

Fine-tuning is further training an existing LLM on many examples of a specific task, so its behaviour changes permanently for that task.

	Prompting	RAG	Fine-tuning
Changes	The request	What the model reads	The model itself
Good for	Most tasks	Your documents, current facts	Consistent style or format; specialised language
Needs	A clear prompt	A document collection and search	Hundreds to thousands of examples
Effort and cost	Low	Medium	Higher
Update facts?	Edit the prompt	Update the documents	Retrain

Rule: fine-tune for behaviour (style, format, specialised language); use RAG for knowledge that changes.

AI agents and tool use

An AI agent is an LLM that works towards a goal in a loop: it decides on an action, uses a tool such as search or code execution, looks at the result, and repeats until done.

1 Goal	2 Decide	3 Act	4 Observe
Which ward had most readmissions, and why?	Plan the next action.	Call a tool: SQL query, Python, search.	Read the result; repeat or finish.

↻ **Steps 2 to 4 repeat until the goal is met.**

Connecting tools: shared standards such as the Model Context Protocol (MCP) let one agent use many tools.

Risk: errors compound over steps. Agents that can act (send, delete, pay) need limits and human approval.

Choosing how to use an LLM

Choosing an approach means matching the problem to the simplest way of using an LLM that solves it: prompting, RAG, fine-tuning or an agent.

If you need…	Start with
A quick answer, draft, summary or explanation	Prompting
A strict format, a style, or step-by-step reasoning	Prompting techniques
Answers from your own or up-to-date documents	RAG
The same specialised task done thousands of times	Fine-tuning
A multi-step task using data, code or other tools	An agent, with limits

Simplest first: each step up adds effort and risk. Combine them when needed, for example an agent that uses RAG.